

TheKernel 操作系统设计与实现

2026年操作系统设计赛“内核实现”赛道

TheKernel Rust Linux-compatible kernel for RISC-V64 and LoongArch64

陈天意 DOTADIW / T202610346999652

杭州师范大学 (HZNU)

全国大学生计算机系统能力大赛操作系统设计赛

2026 年 6 月 30 日

系统实现重点

Linux 状态对象化

进程、线程、地址空间、fd、open file description 和网络 namespace 都有稳定所有者，不靠 syscall 入口临时拼状态。

关键路径集中化

syscall restart、fast path、COW 缺页、poll/epoll 和 socket I/O 都收敛到少数共享路径。

架构差异集中处理

RISC-V64 与 LoongArch64 的 syscall alias、用户上下文、restart PC、页表和 virtio 差异被压在边界层。

测例路径对应对象

clone、exec、mmap、futex、signal、timer、socket 等负载能映射回具体对象和控制流。

系统定位

TheKernel 是面向 OSComp 内核实现赛道的 Rust Linux 兼容内核，在 StarryOS/ArceOS 生态基础上补齐比赛负载需要的进程、内存、文件、网络和双架构运行能力。

运行范围

运行与验证面向 RISC-V64、LoongArch64 的 QEMU virt 环境，覆盖 OSComp 负载触发的 Linux 用户态行为。

- ▶ 可构建：统一生成双架构内核和磁盘镜像。
- ▶ 可运行：guest init 脚本建立测试环境并串行调度负载。
- ▶ 可解释：核心状态都有明确所有者对象。
- ▶ 复测路径：构建、启动、功能组、LTP 和日志摘要串成固定闭环。

从 StarryOS 到 TheKernel

继承层

ArceOS runtime、基础 task 扩展点、signal manager、页表与 VFS 框架仍是底座。

强化层

系统调用分发、任务模型、地址空间和 fd 模型被改成 Linux 可见状态的长期对象。

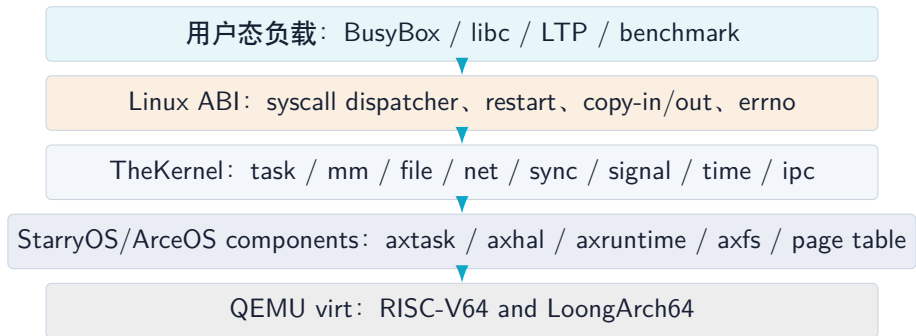
新增层

本地 axnet-ng、mount ledger、BPF surface、扩展 pseudofs 和 OSComp 复测脚本进入仓库。

StarryOS baseline: 2e075accf4fb

TheKernel candidate: d871e3a

系统分层架构



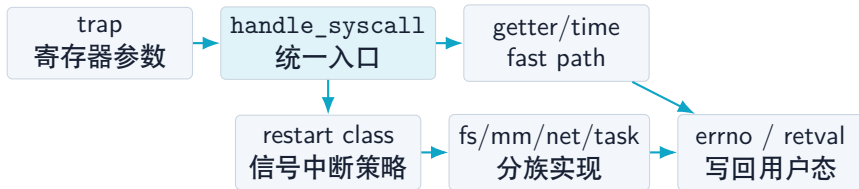
Linux 状态的所有者

对象	负责的 Linux 可见状态
ProcessData	地址空间句柄、fd table、凭证、namespace、进程级 signal、wait/exec/vfork 协调。
Thread	TID、clear-child-tid、robust futex、线程级 signal mask、restart tracker、CPU 记账。
AddrSpace	VMA、Backend、COW/file/shared/linear 缺页填充、fork 与 exec 的地址空间边界。
FileDescription	open file description、status flags、OFD lock owner、异步 I/O owner 和打开凭证。
NetStack	per-namespace listen table、socket set、router/service 和临时端口分配。

机制到测例的映射

测例行为	关键机制	评测关注点
clone/ exec/ wait	ProcessData、Thread、exec/vfork 控制状态、zombie 快照	线程组、父子关系、退出回收、pidfd wait。
mmap/ fork/page fault	AddrSpace+Backend、COW frame 引用、VMA 附加语义	COW 正确性、权限变化、文件映射、清理顺序。
dup/ fcntl/ poll/ epoll	fd table、FileDescription、FileLike 和事件对象	fd-local 与 open-file 状态分离，多路复用一致性。
socket/Unix/TCP/UDP	per-namespace NetStack、Socket enum、fd wrapper	bind/listen/connect、Unix path/abstract、poll 接入。
signal/futex/timer	RestartTracker、futex table、timer manager、timerfd	EINTR/restart、等待取消、deadline 与 clock 语义。

系统调用入口：restart 与 fast path 收敛



- ▶ 双架构 alias、restart bookkeeping 和 fast path 集中在 dispatcher 层。
- ▶ 阻塞调用的中断恢复不分散在各 syscall 文件中临时处理。

fd 模型：统一 special fd 与 poll

fd slot

整数 fd 指向一个槽位。槽位保存 FD_CLOEXEC 这类 fd-local 状态，fork 与 dup 后可以拥有不同槽位。

FileDescription

打开文件描述保存 O_APPEND、O_NONBLOCK、OFD lock owner、lease 与异步 I/O owner，多个 fd 可以共享。



地址空间：Backend 统一 mmap/fork/page fault

COW

fork 后共享只读物理页，写 fault 根据引用数决定提权或复制。

File

文件映射按页填充，处理 EOF 后 gap zeroing 与共享写回边界。

Shared

共享匿名页服务
shm、MAP_SHARED
与多进程可见内存。

Linear

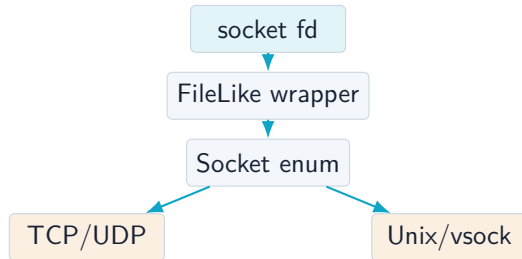
线性映射服务内核控制的连续物理区间。

- ▶ page fault 入口面向 Backend 抽象，VMA 附加语义由 AddrSpace 记录。
- ▶ fork、mmap、mprotect 和 munmap 沿同一生命周期清理资源。

网络栈：NetStack 承接 socket namespace

状态容器

NetStack 保存 listen table、socket set、router/service 和 ephemeral port allocator。普通 clone 共享网络栈，CLONE_NEWNET 创建新栈。



双架构：差异压在边界层

维度	RISC-V64	LoongArch64
syscall ABI	常规编号与寄存器约定，保留 <code>riscv_hwprobe</code> 等架构接口。	<code>time64 alias</code> 更多， <code>dispatcher</code> 映射到共享实现。
restart PC	信号恢复时按架构指令长度回退用户 PC。	同样由架构上下文提供 PC 更新规则。
页表与设备构建产物	QEMU virt 上显式走 <code>virtio-mmio</code> 。 <code>kernel-rv</code> 、 <code>disk-rv.img</code> 。	默认平台配置对应 <code>virtio-pci</code> 。 <code>kernel-la</code> 、 <code>disk-la.img</code> 。

AI Agent 使用边界

环节	AI 参与	人工负责
代码阅读	快速定位模块、提取候选路径。	判断源码事实和抽象边界。
实现修改	生成候选补丁、测试脚本、文档草稿。	选择方案、约束接口、确认行为。
失败分析	阅读日志、提出可能根因。	确认真因，决定修复范围。
验证记录	整理复测步骤和证据。	定义通过标准，承担最终结论。

- ▶ AI 是工程协作工具，不能替代架构决策、验收标准和最终责任。

总结

对象边界明确

Linux 可见状态落到
ProcessData、Thread、
AddrSpace、FileDescription
和 NetStack 等对象上。

控制流集中

syscall restart、COW fault、
fd sharing、socket
namespace 和双架构 alias
都有明确控制流。

运行记录

双架构 replay 记录启动状
态、功能组结果、LTP
deadline 和失败分类。

TheKernel 将比赛负载触发的 Linux 语义收敛到可维护、可复测的内核对象模型。